ASP.NET Core MVC - Full CRUD Documentation

Xenofon Gkioka | Week 2

What We Built

A full CRUD (Create, Read, Update, Delete) system for Categories using:

- ASP.NET Core MVC (.NET 10)
- Entity Framework Core
- SQL Server (Docker)
- Bootstrap 5

Project Structure

```
MyProject/
├── Controllers/
│   ├── CategoryController.cs     ← handles all category requests
│   └── Data/
│       └── ApplicationDbContext.cs  ← bridge to database
├── Models/
│   └── Category.cs               ← data shape / DB table
├── Views/
│   └── Category/
│       ├── Index.cshtml           ← list all categories
│       ├── Create.cshtml          ← create form
│       ├── Edit.cshtml            ← edit form
│       └── Delete.cshtml          ← delete confirmation
```

```
├── Migrations/                    ← auto-generated DB sync files
├── appsettings.json               ← connection string
└── Program.cs                     ← app setup + DI container
```

Step 1 — The Model (Models/Category.cs)

```csharp
public class Category
{
    [Key]                          // Primary Key
    public int Id { get; set; }

    [Required]                     // Cannot be null
    public string? Name { get; set; }

    public int DisplayOrder { get; set; }

    public DateTime CreatedDateTime { get; set; } = DateTime.Now;
}
```

Key concepts:

- `[Key]` → marks the primary key
- `[Required]` → field cannot be empty
- Properties map directly to DB columns

Step 2 — DbContext (Controllers/Data/ApplicationDbContext.cs)

```csharp
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options) { }

    public DbSet<Category> Categories { get; set; }  // = Categories table
}
```

Key concepts:

- Inherits from `DbContext` — gives you all EF Core powers
- Each `DbSet<T>` = one table in the database
- Constructor receives options from DI (connection string etc.)

Step 3 — Register DbContext in Program.cs

```csharp
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")
    )
);
```

Key concepts:

- Registers `ApplicationDbContext` in the DI container
- Reads connection string from `appsettings.json`
- ASP.NET now manages the DB connection lifetime

Step 4 — Migrations

```bash
# Generate migration file
dotnet ef migrations add InitialCreate

# Apply to database
dotnet ef database update
```

Key concepts:

- Migration = snapshot of model changes translated to SQL
- `Up()` → applies the change (CREATE TABLE)
- `Down()` → rolls it back (DROP TABLE)
- Run these every time you change a Model

Step 5 — The Controller (Controllers/CategoryController.cs)

```csharp
```

```
using Microsoft.AspNetCore.Mvc;
using MyProject.Data;
using MyProject.Models;

namespace MyProject.Controllers
{
    public class CategoryController : Controller
    {
        // Dependency Injection — ASP.NET provides the DbContext
        private readonly ApplicationDbContext _db;

        public CategoryController(ApplicationDbContext db)
        {
            _db = db;
        }

        // READ — GET /Category/Index
        public IActionResult Index()
        {
            var objCategoryList = _db.Categories.ToList();
            return View(objCategoryList);
        }

        // CREATE — GET /Category/Create
        public IActionResult Create()
        {
            return View();
        }

        // CREATE — POST /Category/Create
        [HttpPost]
        [ValidateAntiForgeryToken]
        public IActionResult Create(Category obj)
```

```
{
    if (ModelState.IsValid)
    {
        _db.Categories.Add(obj);
        _db.SaveChanges();
        return RedirectToAction("Index");
    }
    return View(obj);
}

// EDIT – GET /Category/Edit/5
public IActionResult Edit(int? id)
{
    if (id == null) return NotFound();
    var category = _db.Categories.Find(id);
    if (category == null) return NotFound();
    return View(category);
}

// EDIT – POST /Category/Edit/5
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Edit(Category obj)
{
    if (ModelState.IsValid)
    {
        _db.Categories.Update(obj);
        _db.SaveChanges();
        return RedirectToAction("Index");
    }
    return View(obj);
}
```

```
        // DELETE — GET /Category/Delete/5
        public IActionResult Delete(int? id)
        {
            if (id == null) return NotFound();
            var category = _db.Categories.Find(id);
            if (category == null) return NotFound();
            return View(category);
        }

        // DELETE — POST /Category/Delete/5
        [HttpPost, ActionName("Delete")]
        [ValidateAntiForgeryToken]
        public IActionResult DeletePOST(int? id)
        {
            var category = _db.Categories.Find(id);
            if (category == null) return NotFound();
            _db.Categories.Remove(category);
            _db.SaveChanges();
            return RedirectToAction("Index");
        }
    }
}
```

Step 6 — Views

Index.cshtml (list all)

```
cshtml
```

```
@model IEnumerable<Category>
@
{
ViewData
[
"Title"
]

=

"Category List"
;
}


<div class="container mt-4">
    <div class="d-flex justify-content-between mb-3">
        <h2>Category List</h2>
        <a asp-controller="Category" asp-action="Create" class="btn btn-primary
            + Create New Category
        </a>
    </div>
    <table class="table table-bordered table-striped">
        <thead>
            <tr>
                <th>ID</th><th>Name</th><th>Display Order</th><th>Actions</th>
            </tr>
        </thead>
        <tbody>

@foreach

(
```

```
var
 item
in
 Model
)

{


<
tr
>


<
td
>
@
item
.
Id
</
td
>


<
td
>
@
item
.
Name
```

```
</
td
>


<
td
>
@
item
.
DisplayOrder
</
td
>


<
td
>


<
a

asp-action
=
"
Edit
"

asp-route-id
=
```

```
"
@item.Id
"

class
=
"
btn btn-warning
"
>
Edit
</
a
>



<
a

asp-action
=
"
Delete
"

asp-route-id
=
"
@item.Id
"

class
=
```

```
"
btn btn-danger
"
>
Delete
</
a
>



</
td
>



</
tr
>



}

        </tbody>
    </table>
</div>
```

Create.cshtml

```cshtml
```

```
@model Category
<form asp-action="Create" method="post">
    <input asp-for="Id" type="hidden"/>
    <label>Name</label>
    <input asp-for="Name" class="form-control"/>
    <label>Display Order</label>
    <input asp-for="DisplayOrder" class="form-control"/>
    <button type="submit" class="btn btn-primary">Create</button>
</form>
```

Edit.cshtml

cshtml

```
@model Category
<form asp-action="Edit" method="post">
    <input asp-for="Id" type="hidden"/>     ← needed to know which record to up
    <label>Name</label>
    <input asp-for="Name" class="form-control"/>
    <label>Display Order</label>
    <input asp-for="DisplayOrder" class="form-control"/>
    <button type="submit" class="btn btn-primary">Update</button>
</form>
```

Delete.cshtml

cshtml

```
@model Category
<form asp-action="Delete" method="post">
    <input asp-for="Id" type="hidden"/>     ← needed to know which record to de
    <p>Name: @Model.Name</p>
    <p>Display Order: @Model.DisplayOrder</p>
    <button type="submit" class="btn btn-danger">Delete</button>
    <a asp-action="Index" class="btn btn-secondary">Cancel</a>
</form>
```

EF Core Quick Reference

```csharp
_db.Categories.ToList()        // get all
_db.Categories.Find(id)        // get by PK
_db.Categories.Add(obj)        // insert
_db.Categories.Update(obj)     // update
_db.Categories.Remove(obj)     // delete
_db.SaveChanges()              // COMMIT — always call after changes
```

Common Mistakes

Mistake	Fix
Forgot `_db.SaveChanges()`	Data staged but never saved to DB
`DeletePOST` name conflict	Use `[ActionName("Delete")]` on POST

Mistake	Fix
`return View(category)` outside variable scope	Declare variable before if blocks
Wrong namespace casing (`Myproject` vs `MyProject`)	Match exactly — C# is case sensitive
Default arg on form button type	Always `type="submit"`, never `type="delete"`